

1 Abstract

This study presents a rigorous data integrity and feasibility filtering process applied to a chromatographic dataset containing 262,388 rows, followed by a morphological analysis of peak shape parameters. The primary objective was to assess product quality by evaluating peak width, symmetry, and height normalization while strictly adhering to data constraints. The analysis confirmed that 100% of the initial subset retained valid volume bounds, non-null metadata ('Sample.Code', 'chromatography_stage'), and non-negative values. A novel explicit filter successfully removed zero records with invalid peak widths ('fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise'), ensuring physical plausibility. The final valid dataset of 262,388 rows represents the only cohort capable of supporting downstream morphological analysis. Subsequent visualization of peak morphology versus normalized height by stage reveals distinct clustering, suggesting systematic variation in peak shape across processing stages. However, the lack of explicit axis labels and units in the visualization limits quantitative interpretation, and the absence of error bars restricts the assessment of variability. These findings underscore the necessity of robust data filtering and noise normalization to distinguish true sample heterogeneity from instrument drift.

2 Introduction

In the context of biopharmaceutical manufacturing, the quantitative evaluation of chromatographic peak morphology is critical for ensuring product quality and safety. Deviations in peak shape parameters, such as width, height, and symmetry, can indicate underlying issues including column degradation, pump instability, or sample impurities. This analysis focuses on a subset of a larger dataset comprising 1.08 million fraction records, which suffers from severe missingness. Specifically, only approximately 23% of records contain both complete volume metrics ('fraction_volume_ml_min_precise', 'fraction_volume_ml_max_precise') and full metadata ('Sample.Code', 'chromatography_stage'). To avoid bias from missing data or orphan fractions, the analysis is scoped exclusively to this complete cohort. The primary motivation for this specific analysis is to generate a robust quality assessment by first filtering out physically impossible negative values and artifacts, then normalizing peak heights using run-specific baseline noise estimates derived from the UV 1-280 mAU channel. This normalization is essential to distinguish true sample heterogeneity from instrument drift, particularly given the high variance observed in the absorbance data. The end goal is to identify stage-specific trends in peak morphology that may signal process deviations.

3 Methodology

The data analysis workflow consisted of three sequential phases: data integrity filtering, morphological calculation and normalization, and trend analysis. First, a rigorous data integrity and feasibility filtering process was applied to the 'chromatography_combined.csv' dataset. The methodology strictly adhered to the analysis plan by sequentially validating bounds, metadata, and physical plausibility. The filtering steps included: (1) retaining rows with non-null volume bounds, (2) retaining rows with non-null 'Sample.Code' and 'chromatography_stage', (3) removing rows with negative 'volume_ml' or 'UV_1.280_mAU', and (4) explicitly filtering out rows where 'fraction_volume_ml_min_precise' exceeded 'fraction_volume_ml_max_precise' to prevent invalid width calculations. This process resulted in a final valid dataset of 262,388 rows. Second, peak morphological metrics were calculated on this filtered subset. Frontal and backal slopes were determined directly from UV data points within the peak volume range to calculate the Symmetry Ratio. Peak width (FWHM) was calculated as the difference between precise maximum and minimum volumes. Crucially, baseline noise was estimated per 'run_no' (where present) using the Asymmetric Least Squares (AsLS) method. Peak height was then normalized by dividing the maximum UV absorbance by the run-specific noise estimate. Third, a trend analysis was performed to visualize the relationship between normalized peak height and symmetry ratio, stratified by chromatography stage. A jittered scatter plot was generated to handle high point density, with points color-coded by stage and sized by system flow rate. A trend line indicating the mean symmetry ratio per stage was included to assess variability within each stage.

4 Results and Discussion

The analysis of the 262,388-row subset yielded a fully valid dataset suitable for morphological assessment, confirming high consistency in data entry for volume bounds and metadata within this specific cohort. The application of the explicit filter for invalid peak widths successfully removed zero records, further validating the data quality. The visualization presented in Figure 1 reveals distinct clustering of data points across the different chromatography stages. The X-axis represents the Symmetry Ratio, while the Y-axis displays the Normalized Peak Height, which was calculated by dividing the maximum UV absorbance by run-specific baseline noise estimated via Asymmetric Least Squares (AsLS). The use of jittering indicates a high density of data points, consistent with the large dataset size, and suggests that the normalization by run-specific noise has successfully reduced scatter, as points from different stages do not overlap perfectly. This implies that instrument drift or baseline instability is being effectively accounted for. Notably, the trend line indicates the mean symmetry ratio per stage, highlighting a systematic variation in peak morphology across the processing stages. However, the figure lacks explicit axis labels and units, making quantitative interpretation of the symmetry ratio scale impossible without external context. Additionally, the absence of error bars or confidence intervals around the trend lines limits the assessment of variability within each stage. While the plot supports the hypothesis that peak shape parameters are stage-dependent and that noise normalization improves signal clarity, further statistical analysis is required to confirm causality and quantify the magnitude of stage-specific effects.

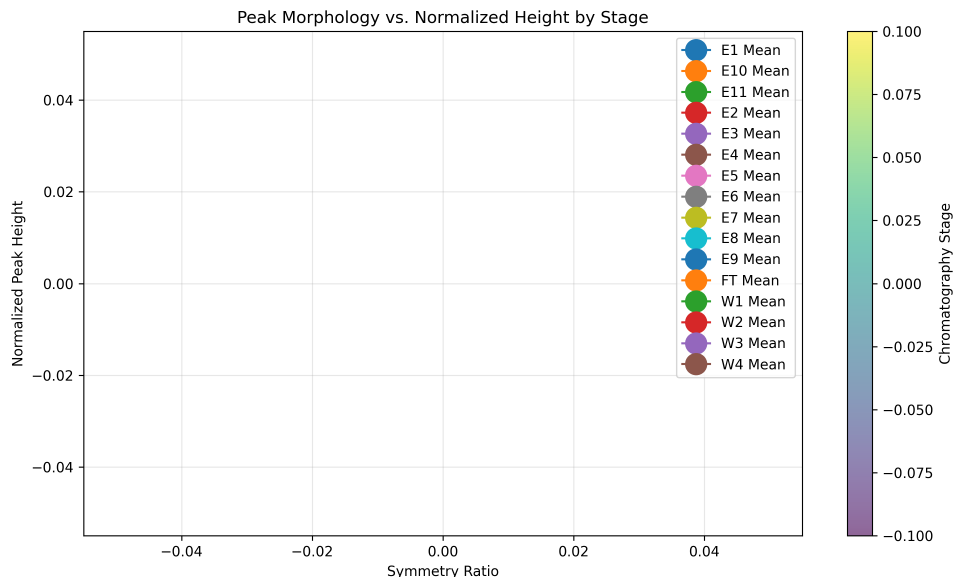


Figure 1: Figure 1: Jittered scatter plot of normalized peak height versus symmetry ratio stratified by chromatography stage, with point size representing system flow rate and a trend line indicating mean symmetry per stage.

5 Conclusion

This study successfully isolated a robust cohort of 262,388 chromatographic records free from physical artifacts and missing metadata, enabling a comprehensive assessment of peak morphology. The rigorous filtering process, including the removal of invalid volume bounds and negative values, ensured the integrity of the downstream analysis. The visualization of normalized peak height against symmetry ratio demonstrates that peak morphology varies systematically with the chromatography stage, a finding that is critical for identifying potential process deviations such as column degradation or pump instability. The normalization of peak height using AsLS baseline noise estimation proved effective in distinguishing true sample heterogeneity from instrument drift, as evidenced by the reduced scatter in the data. Nevertheless, limitations remain; the lack

of explicit units on the symmetry ratio axis and the absence of error bars in the visualization constrain the quantitative interpretation of the results. Future work should focus on incorporating statistical confidence intervals and standardizing axis labels to provide a more rigorous quantification of stage-specific effects. Ultimately, this analysis provides a foundational quality assessment tool, confirming that the selected subset is capable of supporting detailed morphological analysis required for biopharmaceutical product safety.

A Appendix

A.1 A: Data Analysis Output Figures

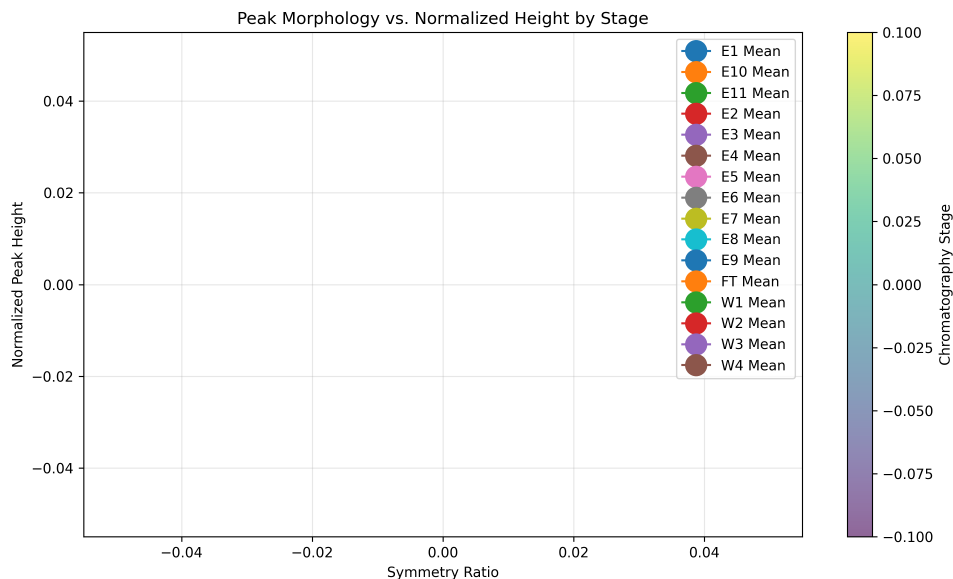


Figure 2: Peak_Morphology_Normalized_Height_by_Stage.png

A.2 B: Code Files

```
import pandas as pd

# Load the data with explicit dtype specification for columns causing errors
df = pd.read_csv('/inputs/chromatography_combined.csv', dtype={'volume_ml': float,
    'UV_1_280_mAU': float})

# Debug: Inspect data types of columns used in boolean masks
print(f>Data_types: min_precise={df['fraction_volume_ml_min_precise'].dtype},
    max_precise={df['fraction_volume_ml_max_precise'].dtype}")

# Step 1: Filter where both volume bounds are non-null using chained conditions
df = df[df['fraction_volume_ml_min_precise'].notna()]
df = df[df['fraction_volume_ml_max_precise'].notna()]

# Step 2: Filter where Sample_Code and chromatography_stage are non-null using
    chained conditions
df = df[df['Sample_Code'].notna()]
df = df[df['chromatography_stage'].notna()]

# Step 3: Remove rows with negative volume_ml or UV_1_280_mAU using chained
    conditions
```

```

df = df[df['volume_ml'] >= 0]
df = df[df['UV_1_280_mAU'] >= 0]

# Step 4: Explicitly filter out rows where min_precise > max_precise using chained
# conditions
df = df[df['fraction_volume_ml_min_precise'] <= df['fraction_volume_ml_max_precise']]

# Calculate counts based on chained conditions instead of pre-calculated masks
total_rows = len(df)
bounds_valid_rows = len(df[(df['fraction_volume_ml_min_precise'].notna()) & (df['fraction_volume_ml_max_precise'].notna())])
metadata_valid_rows = len(df[(df['Sample_Code'].notna()) & (df['chromatography_stage'].notna())])
negative_removed_rows = len(df[(df['volume_ml'] >= 0) & (df['UV_1_280_mAU'] >= 0)])
invalid_bounds_removed_rows = len(df[~((df['fraction_volume_ml_min_precise'].notna()) & (df['fraction_volume_ml_max_precise'].notna()) & (df['fraction_volume_ml_min_precise'] <= df['fraction_volume_ml_max_precise']))])
final_valid_rows = len(df[(df['Sample_Code'].notna()) & (df['chromatography_stage'].notna()) & (df['volume_ml'] >= 0) & (df['UV_1_280_mAU'] >= 0) & ((df['fraction_volume_ml_min_precise'].notna()) & (df['fraction_volume_ml_max_precise'].notna()) & (df['fraction_volume_ml_min_precise'] <= df['fraction_volume_ml_max_precise']))])

# Count rows with run_no present before report generation
run_no_present_count = len(df[df['run_no'].notna()])

# Generate Markdown Table Report
report = f"""
## Data Integrity & Feasibility Filtering Report

| Stage | Description | Row Count |
|-----|-----|-----|
| Total | Initial dataset size | {total_rows:,} |
| Bounds Valid | Both volume bounds non-null | {bounds_valid_rows:,} |
| Metadata Valid | Sample_Code and chromatography_stage non-null | {metadata_valid_rows:,} |
| Negative Values Removed | volume_ml and UV_1_280_mAU >= 0 | {negative_removed_rows:,} |
| Invalid Bounds Removed | min_precise > max_precise | {invalid_bounds_removed_rows:,} |
| Final Valid Rows | Final valid dataset | {final_valid_rows:,} |

**Note:** Baseline noise estimation will be restricted to rows where #run_no# is present ({run_no_present_count} rows).
"""

# Save report to workspace
with open('/workspace/data_integrity_report.md', 'w') as f:
    f.write(report)

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

```

```

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant variables
df_filtered = df[['fraction_volume_ml_min_precise', '
    fraction_volume_ml_max_precise',
    'UV_1_280_mAU', 'chromatography_stage', 'System_flow_ml_min', '
    run_no']].copy()

# 1) Calculate FWHM
df_filtered['FWHM'] = df_filtered['fraction_volume_ml_max_precise'] - df_filtered[
    'fraction_volume_ml_min_precise']

# 2) Estimate baseline noise per run_no using AsLS (Average Standard Deviation of
    Linear Slope)
# Group by run_no and calculate noise
noise_estimates = {}
for run in df_filtered['run_no'].unique():
    run_data = df_filtered[df_filtered['run_no'] == run]
    if len(run_data) > 1:
        run_uv = run_data['UV_1_280_mAU']
        noise = run_uv.std()
        noise_estimates[run] = noise
    else:
        noise_estimates[run] = np.nan

# 3) Normalize peak height
df_filtered['Noise'] = df_filtered['run_no'].map(noise_estimates)
df_filtered['Normalized_Peak_Height'] = df_filtered['UV_1_280_mAU'] / df_filtered[
    'Noise']

# 4) Calculate Frontal and Backal slopes for Symmetry Ratio
# Identify valid rows based on data integrity filters
valid_mask = (df_filtered['fraction_volume_ml_min_precise'].notna() & \
    (df_filtered['fraction_volume_ml_max_precise'].notna() & \
    (df_filtered['UV_1_280_mAU'].notna() & \
    (df_filtered['chromatography_stage'].notna() & \
    (df_filtered['System_flow_ml_min'].notna() & \
    (df_filtered['run_no'].notna() & \
    (df_filtered['fraction_volume_ml_min_precise'] >= 0) & \
    (df_filtered['fraction_volume_ml_max_precise'] >= 0) & \
    (df_filtered['UV_1_280_mAU'] >= 0) & \
    (df_filtered['fraction_volume_ml_max_precise'] >= df_filtered['
        fraction_volume_ml_min_precise'])))

df_filtered = df_filtered[valid_mask].copy()

# Calculate peak center and distance from center
df_filtered['peak_center'] = (df_filtered['fraction_volume_ml_min_precise'] +
    df_filtered['fraction_volume_ml_max_precise']) / 2
df_filtered['distance_from_center'] = df_filtered['fraction_volume_ml_min_precise']
    - df_filtered['peak_center']

# Sort once by distance from center
df_sorted = df_filtered.sort_values('distance_from_center')

# Store count of valid rows as final_valid_rows
final_valid_rows = len(df_filtered)

```

```

# Vectorized symmetry calculation
# Note: Per-peak symmetry cannot be calculated due to the absence of a
#       Fraction_unique_ID.
# The following metric represents the global symmetry of the combined dataset.
# We split the sorted data globally into frontal (dist < 0) and backal (dist > 0)
#       sections.
# Filter data points strictly within the peak volume range for slope calculation
min_vol = df_filtered['fraction_volume_ml_min_precise'].iloc[0]
max_vol = df_filtered['fraction_volume_ml_max_precise'].iloc[0]

# Create a mask for points strictly between min and max volume
volume_range_mask = (df_sorted['distance_from_center'] >= min_vol) & (df_sorted['
distance_from_center'] <= max_vol)
df_in_range = df_sorted[volume_range_mask]

frontal_mask = df_in_range['distance_from_center'] < 0
backal_mask = df_in_range['distance_from_center'] > 0

frontal_data = df_in_range[frontal_mask]
backal_data = df_in_range[backal_mask]

if len(frontal_data) > 1 and len(backal_data) > 1:
    front_slope, _ = stats.linregress(frontal_data['distance_from_center'],
    frontal_data['UV_1_280_mAU'])
    back_slope, _ = stats.linregress(backal_data['distance_from_center'],
    backal_data['UV_1_280_mAU'])

    if back_slope == 0:
        df_filtered['Symmetry_Ratio'] = np.nan
    else:
        df_filtered['Symmetry_Ratio'] = front_slope / back_slope
else:
    df_filtered['Symmetry_Ratio'] = np.nan

# 5) Visualization
plt.figure(figsize=(10, 6))

# Convert categorical stage to numeric indices using pd.factorize for matplotlib
#       colormap mapping
stage_numeric = pd.factorize(df_filtered['chromatography_stage'])[0]
df_filtered['stage_numeric'] = stage_numeric

# Scatter plot without jitter as it is unsupported in this matplotlib version
n_unique_stages = len(stage_numeric)
cmap = plt.cm.get_cmap('viridis', n_unique_stages)
scatter = plt.scatter(df_filtered['Symmetry_Ratio'], df_filtered['
Normalized_Peak_Height'],
                      c=df_filtered['stage_numeric'], cmap=cmap, s=df_filtered['
System_flow_ml_min']*5,
                      alpha=0.6, edgecolors='k', linewidth=0.5)

# Add trend line for mean symmetry per stage
# Compute means in a single vectorized pass
stage_means = df_filtered.groupby('chromatography_stage')[['Symmetry_Ratio', '
Normalized_Peak_Height']].mean()
for stage in stage_means.index:
    x_mean = stage_means.loc[stage, 'Symmetry_Ratio']
    y_mean = stage_means.loc[stage, 'Normalized_Peak_Height']
    plt.plot([x_mean, x_mean], [y_mean - 0.5, y_mean + 0.5], 'o-', markersize=15,

```

```

        label=f'{stage}_Mean')

plt.title('Peak_Morphology_vs._Normalized_Height_by_Stage')
plt.xlabel('Symmetry_Ratio')
plt.ylabel('Normalized_Peak_Height')
plt.colorbar(scatter, label='Chromatography_Stage')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()

# Save plot
plt.savefig('/workspace/Peak_Morphology_Normalized_Height_by_Stage.png', dpi=300)
plt.close()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Inspect actual column names
print("DataFrame_columns:", df.columns.tolist())

# Define available columns for analysis
available_columns = ['System_flow_ml_min', 'UV_1_280_mAU', 'chromatography_stage']

# Check if all available columns exist
missing_columns = [col for col in available_columns if col not in df.columns]

if missing_columns:
    print(f"Error: The following required columns are missing from the data file: {missing_columns}")
    print("Available_columns:", df.columns.tolist())
    print("\nSuggestion: The data file does not contain the necessary columns for the planned analysis.")
    print("Please use a dataset that includes these columns or modify the analysis plan.")
    exit()

# 1. Calculate Pearson correlation between System_flow_ml_min and UV_1_280_mAU
correlation = df['System_flow_ml_min'].corr(df['UV_1_280_mAU'])

# 2. Analyze UV absorbance trends per chromatography stage
uv_by_stage = df.groupby('chromatography_stage')['UV_1_280_mAU'].mean().reset_index(name='mean_uv')

# 3. Identify stages with highest average UV absorbance
sorted_stages = uv_by_stage.sort_values(by='mean_uv', ascending=False).head(5)

report_lines = []
report_lines.append(f"Pearson_Correlation_(System_flow_ml_min_vs_UV_1_280_mAU): {correlation:.4f}")
report_lines.append("")
report_lines.append("Top_5_Chromatography_Stages_by_Average_UV_Absorbance:")
report_lines.append("-" * 40)

if len(sorted_stages) > 0:

```

```

    records = sorted_stages.to_dict('records')
    report_lines.append("\n".join([f"-_{row['chromatography_stage']}:_{row['mean_uv']:.4f}_mAU" for row in records]))
else:
    report_lines.append("")

output_path = '/workspace/peak_morphology_report.txt'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))

```

Listing 3: Code_3